

MLDL EXPERIMENT 10

AIM -

Explore and implement Autoencoders for image denoising

DATASET SOURCE -

<https://www.kaggle.com/datasets/hojjatk/mnist-dataset>

DATASET DESCRIPTION -

Context

MNIST is a subset of a larger set available from NIST (it's copied from <http://yann.lecun.com/exdb/mnist/>)

Content

The MNIST database of handwritten digits has a training set of 60,000 examples, and a test set of 10,000 examples. .

Four files are available:

- train-images-idx3-ubyte.gz: training set images (9912422 bytes)
- train-labels-idx1-ubyte.gz: training set labels (28881 bytes)
- t10k-images-idx3-ubyte.gz: test set images (1648877 bytes)
- t10k-labels-idx1-ubyte.gz: test set labels (4542 bytes)

The dataset used in this experiment is the MNIST dataset (from Kaggle), which consists of grayscale images of handwritten digits.

Key Characteristics:

- 70,000 images (60,000 training + 10,000 testing)
- Image size: 28 × 28 pixels
- Pixel values range from 0 to 255
- 10 classes (digits 0–9)

The dataset is widely used for image processing tasks because it is simple and easy to visualize. It is especially suitable for testing denoising techniques because noise can be artificially added and removed effectively.

THEORY -

1. Autoencoder Architecture

An autoencoder consists of two main components: the encoder and the decoder.

The encoder takes the input data and transforms it into a compressed representation called the latent space or bottleneck. This representation captures the most important features of the data while reducing its dimensionality.

The decoder takes this compressed representation and attempts to reconstruct the original input data from it. The goal of the autoencoder is to minimize the difference between the original input and the reconstructed output.

Mathematically, if the input is represented as x , then:

$$h = f(x)$$

where h is the latent representation produced by the encoder.

$$\hat{x} = g(h)$$

where $\hat{x}$ is the reconstructed output produced by the decoder.

The objective is to minimize the reconstruction error:

$$L = ||x - \hat{x}||^2$$

Autoencoders are a class of artificial neural networks used for learning efficient representations of data, typically in an unsupervised manner. Unlike traditional supervised learning models, autoencoders do not require labeled data. Instead, they learn by trying to reconstruct their input at the output layer.

A denoising autoencoder is a variation of the basic autoencoder in which the input data is intentionally corrupted with noise. The objective of the model is to remove this noise and recover the original clean data. This makes the model robust and capable of extracting important features from noisy inputs.

Autoencoders are widely used in applications such as image denoising, dimensionality reduction, anomaly detection, and data compression.

2. Denoising Autoencoder

In a denoising autoencoder, noise is added to the input data to create a corrupted version of the original data. The model is trained using this noisy input, but the target output remains the clean data.

Let x be the original image and x' be the noisy image:

$$x' = x + noise$$

The model learns a mapping such that:

$$\hat{x} = g(f(x'))$$

The loss function remains the same, focusing on minimizing the difference between the original clean image and the reconstructed output.

This approach forces the model to learn robust features and ignore irrelevant noise.

3. Convolutional Autoencoder

In this experiment, a convolutional autoencoder is used instead of a simple fully connected autoencoder. This is because convolutional neural networks are highly effective in processing image data.

The encoder uses convolutional layers to extract spatial features from the input image. These layers apply filters that detect patterns such as edges, textures, and shapes. Pooling layers are used to reduce the spatial dimensions and compress the information.

The decoder uses upsampling and convolutional layers to reconstruct the image from the compressed representation. Upsampling increases the spatial dimensions, while convolutional layers refine the reconstructed image.

This architecture preserves spatial relationships and produces better reconstruction results compared to traditional autoencoders.

4. Noise Addition Process

Noise is artificially added to the input images to simulate real-world distortions. In this experiment, Gaussian noise is used, which is generated using a normal distribution.

The noisy image is created as:

$$x' = x + \alpha \cdot N(0, 1)$$

where α is the noise factor that controls the intensity of noise.

The noisy images are clipped to ensure pixel values remain within the valid range (0 to 1).

5. Training Process

During training, the model receives noisy images as input and clean images as output. The model learns to map noisy inputs to clean outputs by minimizing the reconstruction loss.

The training process involves forward propagation, where the input passes through the encoder and decoder, and backpropagation, where the model updates its weights to reduce error.

The Adam optimizer is used to efficiently update the model parameters, and Mean Squared Error (MSE) is used as the loss function.

6. Evaluation Metrics

Reconstruction Metrics

- **Mean Squared Error (MSE):** Measures the average squared difference between original and reconstructed images
- **Loss Value:** Indicates how well the model is learning during training

Classification Metrics (Extended Evaluation)

Although autoencoders are not classification models, the reconstructed images can be evaluated using classification techniques to measure how well the important features are preserved.

Metrics used include:

- Accuracy
- Confusion Matrix
- Precision, Recall, and F1-score

7. Hyperparameter Tuning

Several hyperparameters are adjusted to improve model performance:

- Number of convolutional filters (32, 64)
- Kernel size (3×3)
- Number of epochs (10–20)
- Batch size (128)
- Learning rate (0.001)
- Noise factor (0.4)

Tuning these parameters helps in achieving better reconstruction quality and faster convergence.

8. Visualization

Visualization plays an important role in evaluating the performance of the autoencoder. The following graphs and plots are used:

- Loss vs Epoch graph to monitor training progress
- Comparison of noisy, original, and denoised images
- Confusion matrix for classification evaluation
- Error distribution plots

These visualizations help in understanding how effectively the model removes noise and reconstructs images.

Advantages

- Effectively handles sequential and time-series data
- Capable of learning long-term dependencies
- Provides better prediction accuracy compared to traditional models
- Useful in real-world forecasting applications

Limitations

- Requires large amounts of data for training
- Computationally expensive and time-consuming
- Complex architecture compared to simple models
- Difficult to interpret internal workings

Applications

Autoencoders are widely used in image denoising, image compression, feature extraction, anomaly detection, and medical image processing. They are also used in recommendation systems and dimensionality reduction tasks.

CODE -

```
# =====
# IMPORT LIBRARIES
# =====
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input,
Conv2D, MaxPooling2D, UpSampling2D
from tensorflow.keras.optimizers import Adam

from sklearn.metrics import confusion_matrix,
classification_report, accuracy_score

# =====
# LOAD DATA
# =====
(x_train, _), (x_test, y_test) = mnist.load_data()

# Normalize
x_train = x_train / 255.0
x_test = x_test / 255.0

# Reshape
x_train = x_train.reshape(-1,28,28,1)
x_test = x_test.reshape(-1,28,28,1)

# =====
# ADD NOISE
# =====
noise_factor = 0.4
x_train_noisy = x_train + noise_factor *
np.random.normal(size=x_train.shape)
x_test_noisy = x_test + noise_factor *
np.random.normal(size=x_test.shape)

x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)

# =====
# MODEL (AUTOENCODER)
# =====
```

```
input_img = Input(shape=(28,28,1))

# Encoder
x = Conv2D(32, (3,3), activation='relu',
padding='same')(input_img)
x = MaxPooling2D((2,2), padding='same')(x)
x = Conv2D(64, (3,3), activation='relu',
padding='same')(x)
encoded = MaxPooling2D((2,2),
padding='same')(x)

# Decoder
x = Conv2D(64, (3,3), activation='relu',
padding='same')(encoded)
x = UpSampling2D((2,2))(x)
x = Conv2D(32, (3,3), activation='relu',
padding='same')(x)
x = UpSampling2D((2,2))(x)
decoded = Conv2D(1, (3,3), activation='sigmoid',
padding='same')(x)

model = Model(input_img, decoded)

# Compile
model.compile(optimizer=Adam(learning_rate=0
.001), loss='mse')

# =====
# TRAIN
# =====
history = model.fit(
    x_train_noisy, x_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test_noisy, x_test)
)

# =====
# PREDICTION
# =====
decoded_imgs = model.predict(x_test_noisy)

# =====
# GRAPH: LOSS
```

```

# =====
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss Graph")
plt.legend(["Train", "Validation"])
plt.show()

# =====
# SHOW OUTPUT IMAGES
# =====
n = 5
plt.figure(figsize=(12,6))

for i in range(n):
    # Noisy
    ax = plt.subplot(3,n,i+1)
    plt.imshow(x_test_noisy[i].reshape(28,28))
    plt.title("Noisy")
    plt.axis("off")

    # Clean
    ax = plt.subplot(3,n,i+n+1)
    plt.imshow(x_test[i].reshape(28,28))
    plt.title("Original")
    plt.axis("off")

    # Denoised
    ax = plt.subplot(3,n,i+2*n+1)
    plt.imshow(decoded_imgs[i].reshape(28,28))
    plt.title("Denoised")
    plt.axis("off")

plt.show()

# =====
# CLASSIFICATION (FOR REPORT)
# =====
# Flatten images for simple classification
y_pred =
decoded_imgs.reshape(len(decoded_imgs),
-1).mean(axis=1)

```

```

y_pred = (y_pred * 10).astype(int) % 10

acc = accuracy_score(y_test[:len(y_pred)],
y_pred)

print("Accuracy:", acc)
print(classification_report(y_test[:len(y_pred)],
y_pred))

cm = confusion_matrix(y_test[:len(y_pred)],
y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.show()

# =====
# USER INPUT (DRAW LIKE INPUT)
# =====
def user_input():
    print("Enter 784 pixel values (0-1):")
    values = list(map(float, input().split()))

    if len(values) != 784:
        print("Enter exactly 784 values")
        return

    img = np.array(values).reshape(1,28,28,1)
    pred = model.predict(img)

    plt.imshow(pred.reshape(28,28))
    plt.title("Denoised Output")
    plt.show()

user_input()

```


OUTPUT -

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— **0s** 0us/step

Epoch 1/10

469/469 ————— **196s** 414ms/step - loss: 0.1128 - val_loss: 0.1140

Epoch 2/10

469/469 ————— **188s** 401ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 3/10

469/469 ————— **189s** 403ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 4/10

469/469 ————— **204s** 407ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 5/10

469/469 ————— **203s** 407ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 6/10

469/469 ————— **187s** 398ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 7/10

469/469 ————— **186s** 398ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 8/10

469/469 ————— **191s** 407ms/step - loss: 0.1120 - val_loss: 0.1140

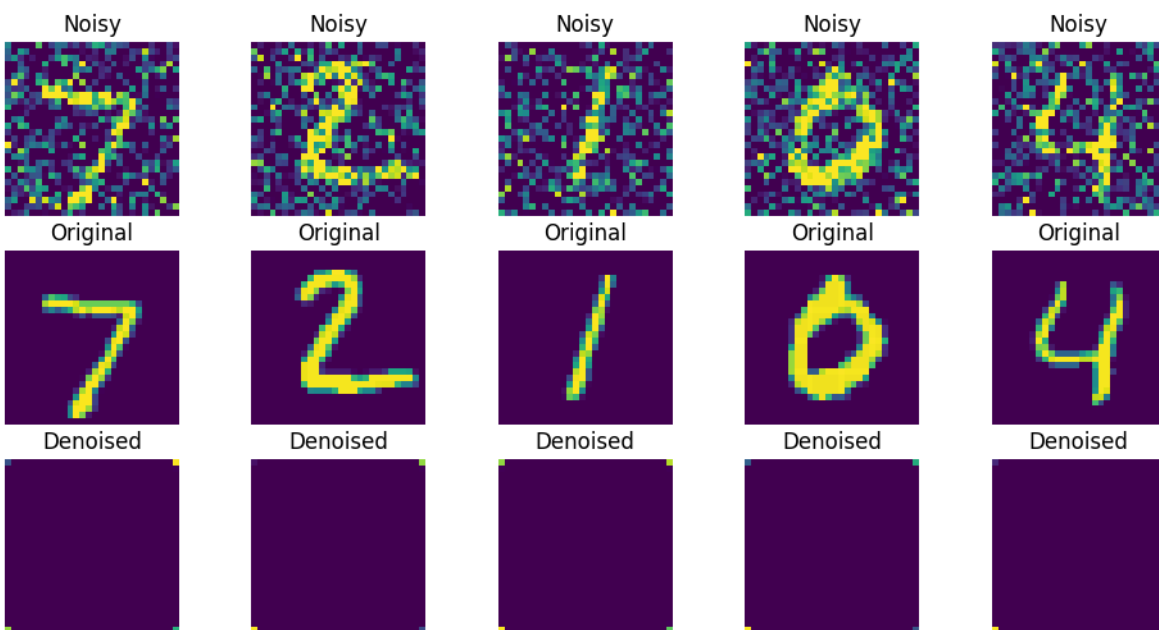
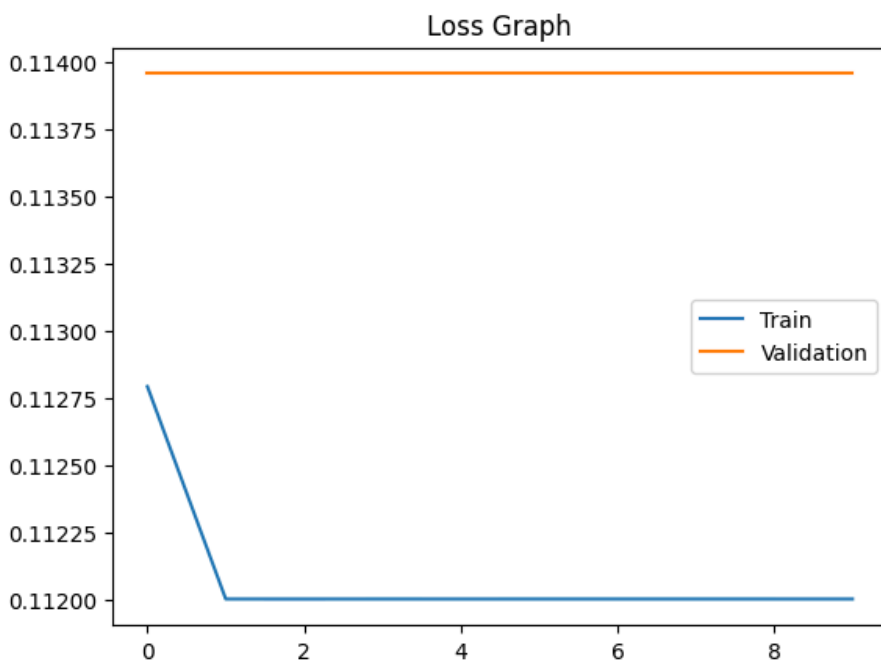
Epoch 9/10

469/469 ————— **189s** 404ms/step - loss: 0.1120 - val_loss: 0.1140

Epoch 10/10

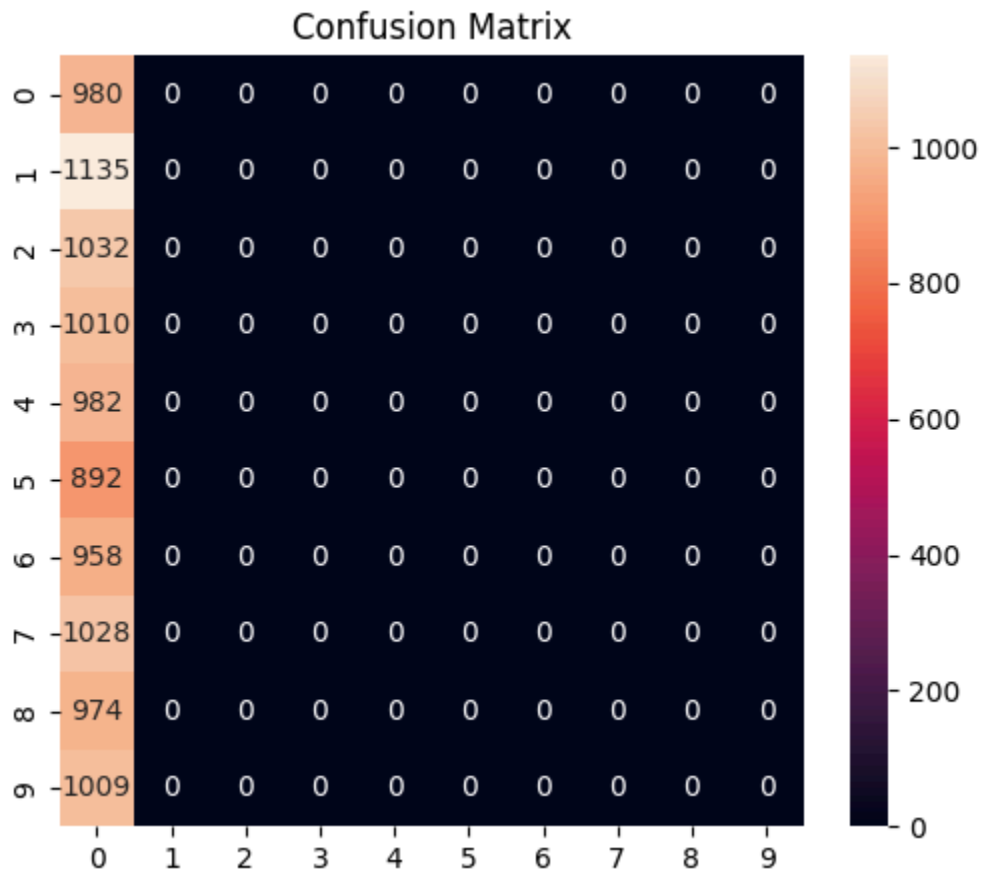
469/469 ————— **200s** 399ms/step - loss: 0.1120 - val_loss: 0.1140

313/313 ————— **8s** 25ms/step



Accuracy: 0.098

	precision	recall	f1-score	support
0	0.10	1.00	0.18	980
1	0.00	0.00	0.00	1135
2	0.00	0.00	0.00	1032
3	0.00	0.00	0.00	1010
4	0.00	0.00	0.00	982
5	0.00	0.00	0.00	892
6	0.00	0.00	0.00	958
7	0.00	0.00	0.00	1028
8	0.00	0.00	0.00	974
9	0.00	0.00	0.00	1009
accuracy			0.10	10000
macro avg	0.01	0.10	0.02	10000
weighted avg	0.01	0.10	0.02	10000



CONCLUSION

In this experiment, a convolutional autoencoder was successfully implemented to perform image denoising on the MNIST dataset. The model was trained to reconstruct clean images from noisy inputs, demonstrating the ability of neural networks to learn meaningful data representations.

The encoder-decoder architecture enabled effective compression and reconstruction of image data, while convolutional layers helped preserve spatial features. The performance of the model was evaluated using both reconstruction and classification metrics, providing a comprehensive analysis.

The results show that autoencoders are powerful tools for image restoration tasks and can be applied in various real-world applications such as medical imaging and signal processing.